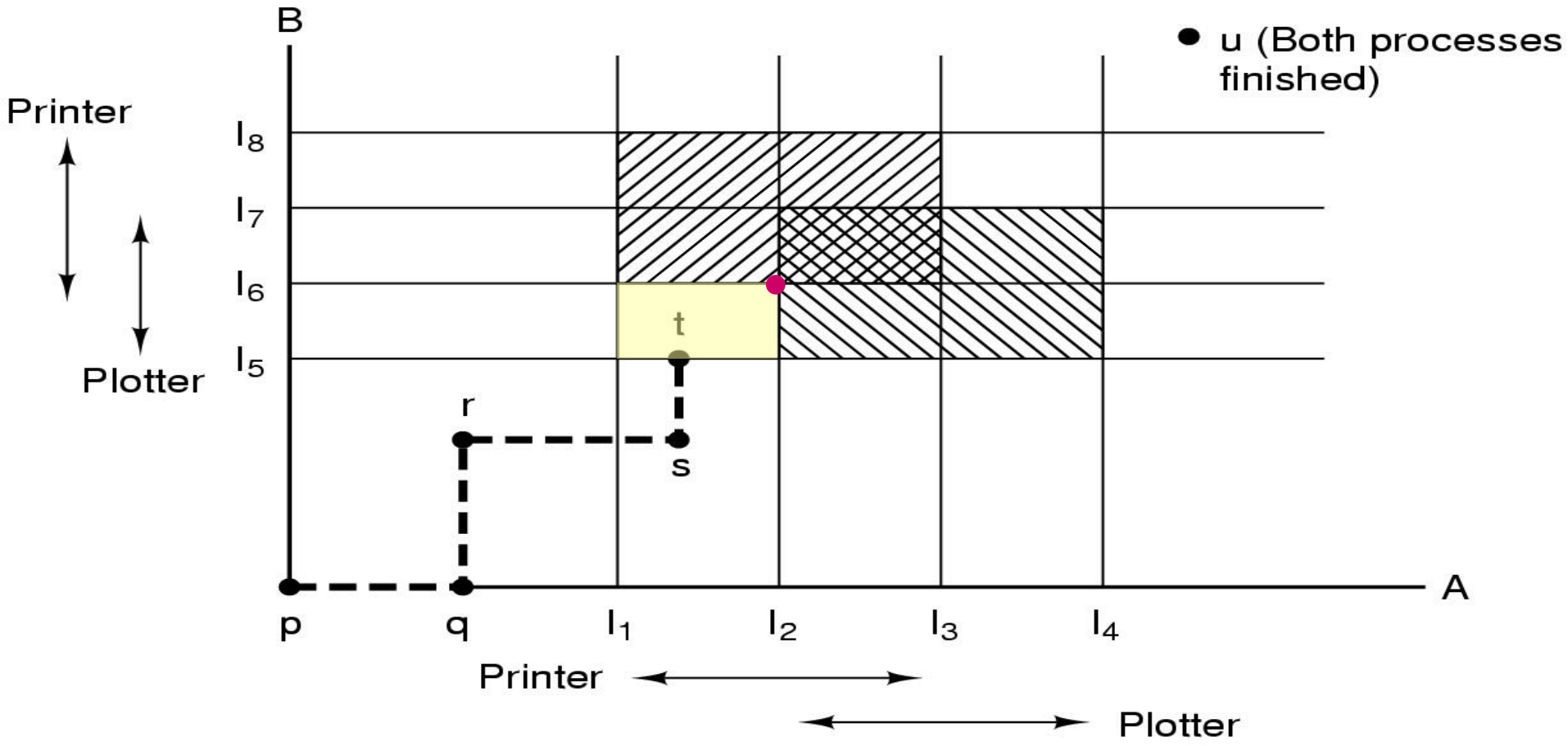
Chapter 6
Deadlocks

6.1 Resource
6.2 Introduction to deadlocks
6.3 The ostrich algorithm
6.4 Deadlock detection and recovery
6.5 Deadlock avoidance
6.6 Deadlock prevention
6.7 Other issues

(3) Deadlock Avoidance

- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Requires that the system has some additional *a priori* information available.
- Simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need.

The deadlock-avoidance algorithm are based on the safe states.

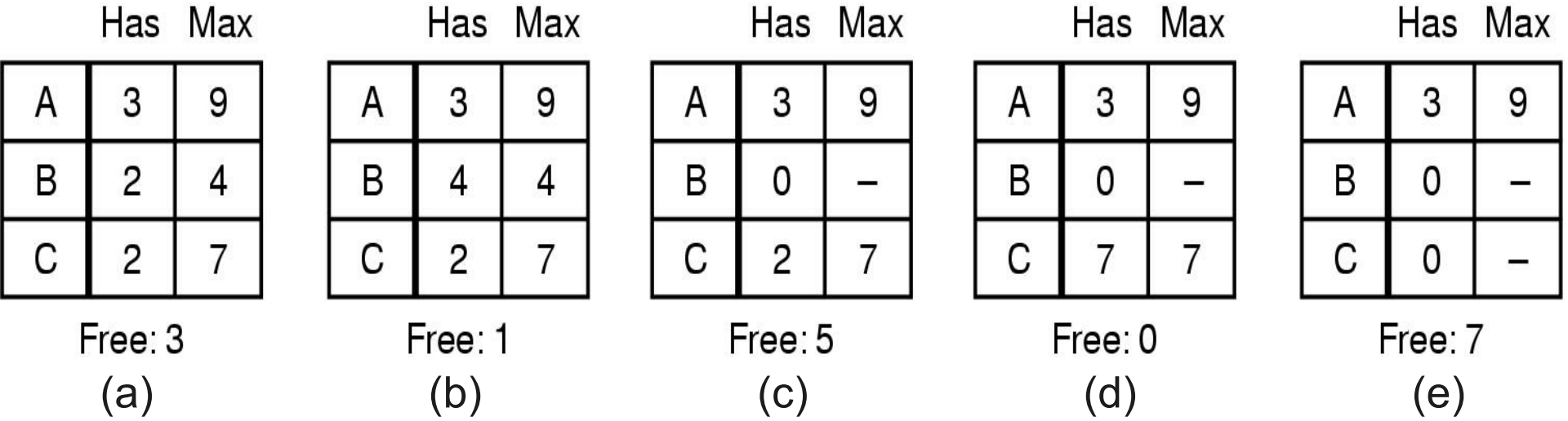


Two process resource trajectories

Safe and Unsafe States

- A state is safe if it is not deadlock and there is a way of satisfying all pending requests by running the process in some order.
- System is in safe state if there exists a safe sequence of all processes.
- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state.

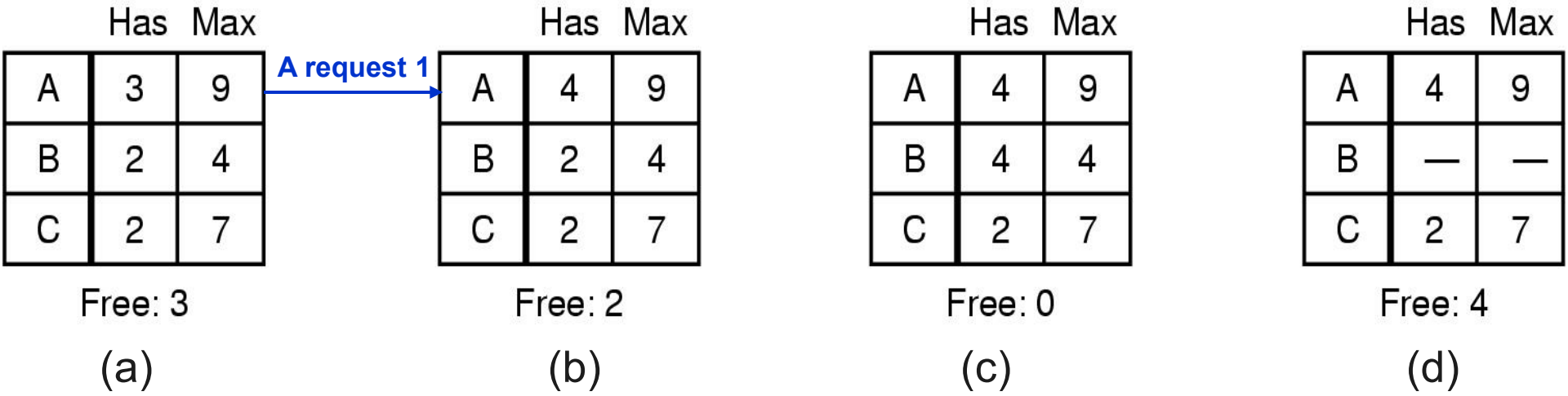
Safe and Unsafe States



Demonstration that the state in (a) is safe

(there exists a safe sequence BCA)

Safe and Unsafe States

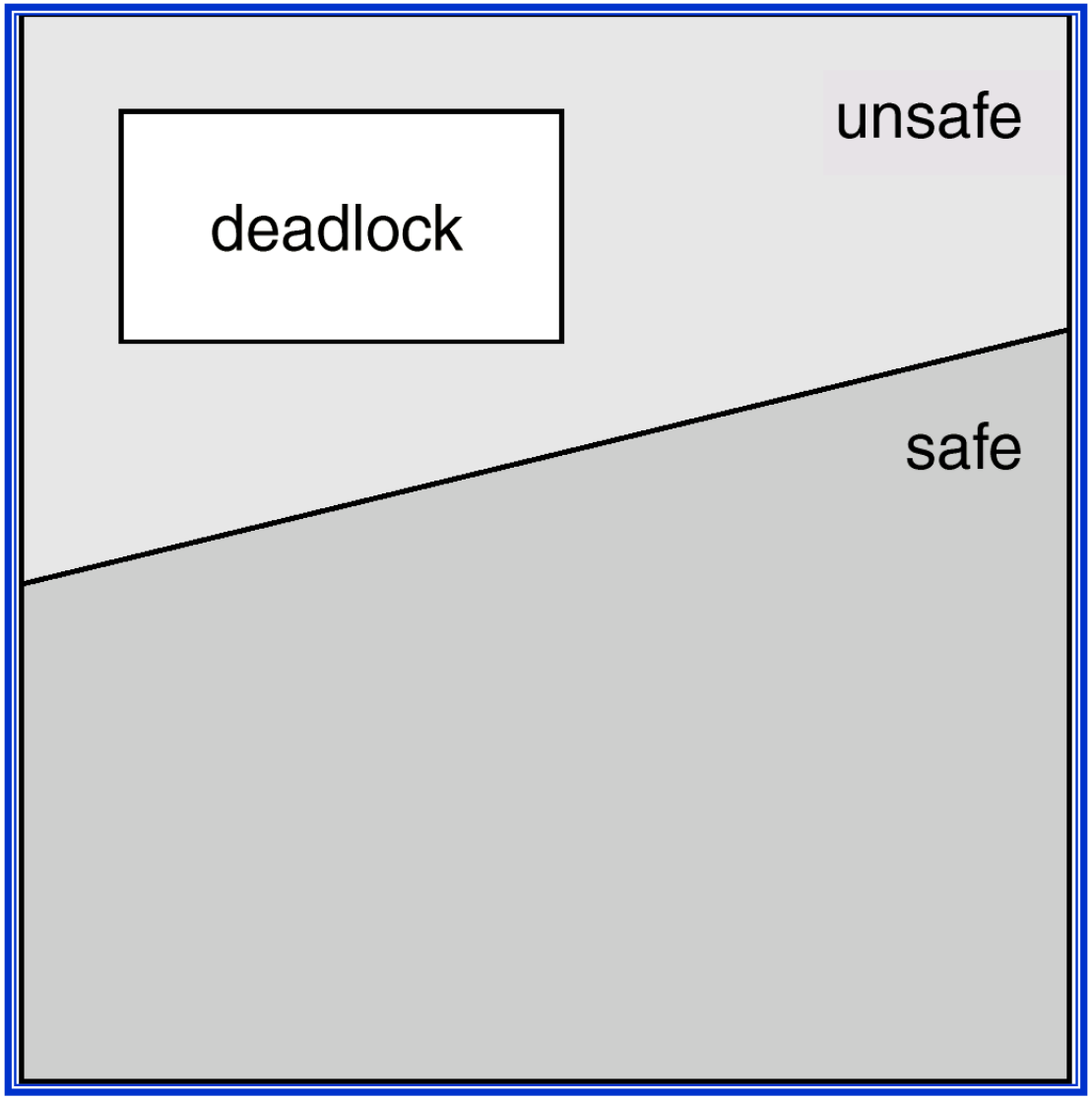


Demonstration that the sate in b is not safe

Basic Facts

- If a system is in safe state $\Rightarrow$ no deadlocks.
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock.
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.

Safe, Unsafe, Deadlock State spaces



Deadlock-avoidance algorithm

- Banker Algorithm (Dijkstra, 1965)
  - Each customer tells banker the maximum number of resources it needs
  - Customer borrows resources from banker
  - Customer returns resources to banker
  - Customer eventually pays back loan
  - Banker only lends resources if the system will be in a *safe* state after the loan
- *Safe state* - there is a lending sequence such that all customers can take out a loan
- *Unsafe state* - there is a possibility of deadlock

The Banker's Algorithm for a Single Resource

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

	Has	Max
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)

B request 1

	Has	Max
A	1	6
B	2	5
C	2	4
D	4	7

Free: 1

(c)

- Three resource allocation states
  - (a) safe
  - (b) safe
  - (c) unsafe

Banker's Algorithm for Multiple Resources

Process	Tape drives	Plotters	Scanners	CD ROMs
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resources assigned

C

Process	Tape drives	Plotters	Scanners	CD ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still needed

R

E = (6342)
P = (5322)
A = (1020)

Example of banker's algorithm with multiple resources

Banker's Algorithm

Check to see if a state is safe:

1. Some look for a new row in R which is smaller than A. If no such row exists the system will eventually deadlock ==> not safe.
2. If such a row exists, the process may finish. Mark that process (row) as terminate and add all of its resources to A.
3. Repeat Steps 1 and 2 until all rows are marked == > safe state

   or not marked == > not safe.

How to Compute Safety

Given:

n kinds of resources

p processes

Set P of processes

struct {resource needs[n], owns[n]} ToDo[p]

resource available[n]

```
while there exists a p in P such that
    for all i (ToDo[p].needs[i] < available[i] )
       { do for all i
            available[i] += ToDo[p].owns[i];
            P = P - p;
       }
```

If P is empty then system is safe.

Is Allocation (1 0 2) to P1 Safe?

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	2	0	0	3	2	2	1	2	2	3	3	2
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

And Run Safety Test: Group Discussion

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	3	0	2	3	2	2	0	2	0	2	3	0
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

If P1 requests max resources, can complete

Allocate to P1, Then

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	3	2	2	3	2	2	0	0	0	2	1	0
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

Release - P1 Finishes

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	0	0	0	3	2	2	3	2	2	5	3	2
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

Now P3 can acquire max resources and release

Release – P3 Finishes

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	0	0	0	3	2	2	3	2	2	7	4	3
P2	3	0	0	9	0	2	6	0	2			
P3	0	0	0	2	2	2	2	2	2			
P4	0	0	2	4	3	3	4	3	1			

Now P4 can acquire max resources and release

Release - P4 Finishes

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	0	0	0	3	2	2	3	2	2	7	4	5
P2	3	0	0	9	0	2	6	0	2			
P3	0	0	0	2	2	2	2	2	2			
P4	0	0	0	4	3	3	4	3	3			

Now P2 can acquire max resources and release

Release - P2 Finishes

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	0	0	0	3	2	2	3	2	2	10	4	5
P2	0	0	0	9	0	2	9	0	2			
P3	0	0	0	2	2	2	2	2	2			
P4	0	0	0	4	3	3	4	3	3			

Now P0 can acquire max resources and release

So P1 Allocation (1 0 2) Is Safe

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	3	0	2	3	2	2	0	2	0	2	3	0
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

There exists a safe sequence: P1, P3, P4, P2, P0

Is allocation (0 2 0) to P0 Safe?

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	3	0	2	3	2	2	0	2	0	2	3	0
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

Try to Allocate 2 B to P0

Run Safety Test

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	3	0	7	5	3	7	2	3	10	5	5
P1	3	0	2	3	2	2	0	2	0	2	1	0
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

No Processes may get max resources and release

So Unsafe State - Do Not Enter

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	3	0	2	3	2	2	0	2	0	2	3	0
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

Return to Safe State and do not allocate resource

P0 Suspended Pending Request

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	5
P1	3	0	2	3	2	2	0	2	0	2	3	0
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

When enough resources become available, P0 can awake

Results

- P0's request for 2 Bs cannot be granted because that would prevent any other process from completing if they need their maximum claim.

Just Because It's Unsafe---

- P0 could have been allocated 2 Bs and a deadlock might not have occurred if:
  - P1 say didn't use its maximum resources but finished using the resources it had

Allocate to P0(0 2 0)

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	3	0	7	5	3	7	2	3	10	5	5
P1	3	0	2	3	2	2	0	2	0	2	1	0
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

P1 can finish its work using the resources it just had

If P1 Doesn't Need Max---

Process	Alloc			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	3	0	7	5	3	7	2	3	10	5	5
P1	0	0	0	3	2	2	3	2	2	5	1	2
P2	3	0	0	9	0	2	6	0	2			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

Then P1 would have finished…

Banker Solution Issues

- Trade-off: The banker's algorithm is conservative --- it reduces parallelism for safety sake
- Not very practicable
  - Processes rarely know advance what their maximum resource needs will be
  - The number of processes is not fixed
  - Resources that were thought to be available can suddenly vanish (tape drive can break)

(4) Deadlock Prevention

- Restrain the ways request can be made.
  1. Mutual Exclusion
  2. Hold and Wait
  3. No Preemption
  4. Circular Wait

1. Attacking the Mutual Exclusion Condition

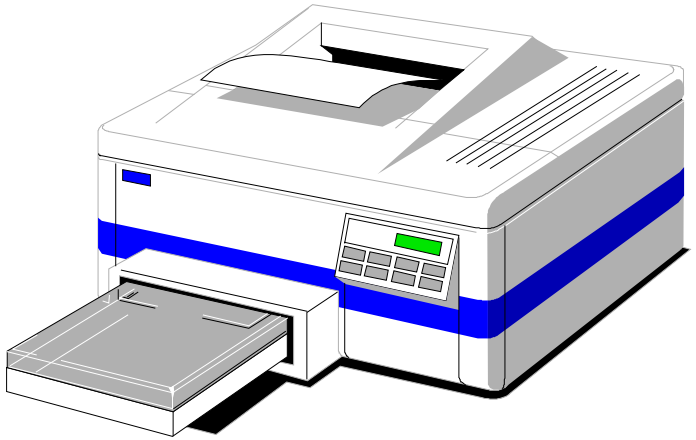
- Some devices (such as printer) can be spooled
  - only the printer daemon uses printer resource
  - thus deadlock for printer eliminated
- Not all devices can be spooled
- Principle:
  - avoid assigning resource when not absolutely necessary
  - as few processes as possible actually claim the resource

2. Attacking the Hold and Wait Condition

- Require processes to request all resources before starting
  - a process never has to wait for what it needs
- Problems
  - may not know required resources at start of run
  - also ties up resources other processes could be using
- Variation:
  - process requesting a resource must give up all the resources it currently holds, then request all immediately needed

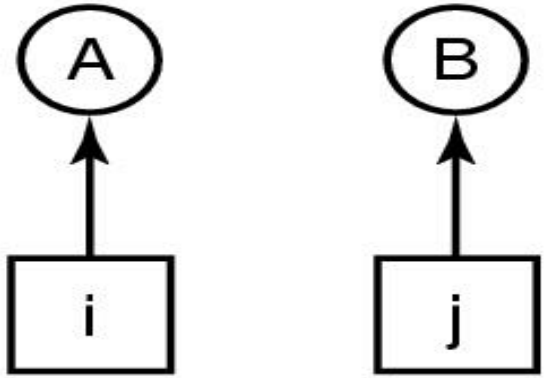
3. Attacking the No Preemption Condition

- This is not a viable option
- Consider a process given the printer
  - halfway through its job
  - now forcibly take away printer
  - !!??



4. Attacking the Circular Wait Condition

1. Imagesetter
2. Scanner
3. Plotter
4. Tape drive
5. CD Rom drive



(a) **(b)**

(a) Normally ordered resources

(b) A resource graph

- Number all resources and require all requests to be made in numerical order.

Summary of approaches to deadlock prevention

Condition	Approach
Mutual exclusion	Spool everything
Hold and wait	Request all resources initially
No preemption	Take resources away
Circular wait	Order resources numerically

Other Issues
Two-Phase Locking

- Phase One
  - process tries to lock all records it needs, one at a time
  - if needed record found locked, start over
- If phase one succeeds, it starts second phase,
  - performing updates
  - releasing locks
- Note similarity to requesting all resources at once
- Algorithm works where programmer can arrange
  - program can be stopped, restarted

Communication Deadlocks

- Resource deadlocks are common, but not the only kind of deadlock.
- Communication deadlocks
  - A set of processes, each blocked waiting for an event only the other one can cause.
  - Example
    - Process A sends a request message to process B, and then blocks until B sends back a reply message.
    - If the request message gets lost, B is blocked waiting for a request asking it to do something.
- Solution: timeout

Livelock

- Polling (busy waiting), used to enter a critical region or access a resource, can lead to livelock
- Example

```
void process_A(void) {
        enter_region(&resource_1);
        enter_region(&resource_2);
        use_both_resources( );
        leave_region(&resource_2);
        leave_region(&resource_1);
}
```

```
void process_B(void) {
        enter_region(&resource_2);
        enter_region(&resource_1);
        use_both_resources( );
        leave_region(&resource_1);
        leave_region(&resource_2);
}
```

  - When A acquires resource_1 and B acquires resource_2, no process can make further progress, and neither process blocks.
    →livelock

Starvation

- Algorithm to allocate a resource
  - may be to give to shortest job first
- Works great for multiple short jobs in a system
- May cause long job to be postponed indefinitely
  - even though not blocked
- Solution:
  - "First-come, first-serve" (FCFS) policy

Deadlock Summary

- In general, deadlock detection or avoidance is expensive
- Must evaluate cost of deadlock against detection or avoidance costs
- Deadlock avoidance and recovery may cause indefinite postponement
- Unix and Windows use Ostrich Algorithm